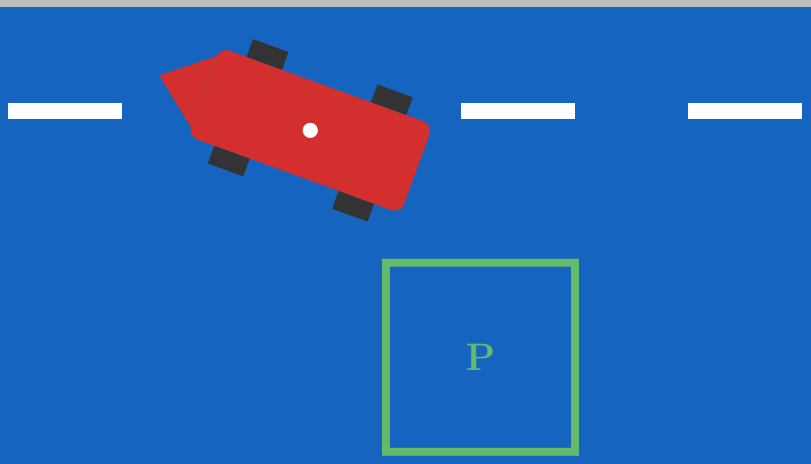
Park Like a Pro

Building a Car Parking Game in Scratch

A game physics adventure for curious minds



6 sessions • Real physics • One awesome game

Welcome!

You know what's really hard? Parking a car. Grown-ups do it every day and sometimes they *still* get it wrong. But here's the secret: parking is just physics — and physics is just a set of rules about how things move.

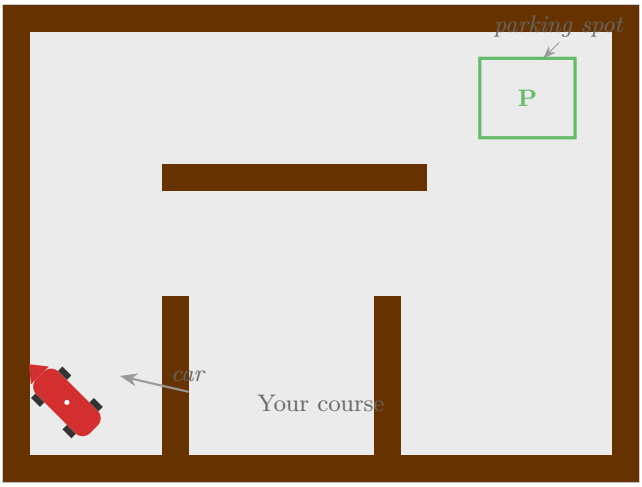
In this tutorial, we're going to learn those rules one at a time. And we're going to use them to build a real game in Scratch — a top-down car parking game where you drive a racing car into a tight parking spot.

By the end, you'll have a game you can show your friends. And you'll understand *why* parking is tricky — which is more than most grown-ups can say.

What you'll need

- A computer with a web browser
- A Scratch account (free at `scratch.mit.edu`)
- A grown-up to read this with (they might learn something too)

What we're building



A top-down racing game where you steer a cigar-shaped race car around a course, avoid the walls, and park it perfectly in a parking spot. There's a timer, a crash counter, and a happy sound when you nail it.

Let's go.

Contents

Welcome **2**

1 Where Is My Car? **4**

1.1 What We Already Know 4

1.2 The New Idea: Position and Direction 4

1.3 Let's Build It 5

1.3.1 Step 1: Create a new project 5

1.3.2 Step 2: Draw your car 5

1.3.3 Step 3: Add movement code . . . 6
1.3.4 Step 4: Try it! . . . 6

2 Fast and Slow **8**
2.1 What We Already Know . . . 8
2.2 The New Idea: Speed . . . 8
2.3 Let's Build It . . . 8
2.3.1 Step 1: Create a speed variable . . . 8
2.3.2 Step 2: Update the movement code . . . 8
2.3.3 Step 3: Add a speed limit . . . 9

3 Turning the Wheel **11**
3.1 What We Already Know . . . 11
3.2 The New Idea: Steering and Turning Radius . . . 11
3.3 Let's Build It . . . 11
3.3.1 Step 1: Create a steering variable . . . 11
3.3.2 Step 2: Update the steering code . . . 12
3.3.3 Step 3: Add a wheel indicator . . . 12

4 Walls and Bumps **14**
4.1 What We Already Know . . . 14
4.2 The New Idea: Collision . . . 14
4.3 Let's Build It . . . 14
4.3.1 Step 1: Design the course . . . 14
4.3.2 Step 2: Add position memory . . . 15
4.3.3 Step 3: Update the code . . . 15

5 Park It! **17**
5.1 What We Already Know . . . 17
5.2 The New Idea: Parking as a Physics Puzzle . . . 17
5.3 Let's Build It . . . 17
5.3.1 Step 1: Create the parking spot . . . 17
5.3.2 Step 2: Add parking detection . . . 18
5.3.3 Step 3: Add a happy sound . . . 19
5.3.4 Step 4: Try it! . . . 19

6 Make It a Real Game **20**
6.1 What We Already Know . . . 20
6.2 The New Idea: The Game Loop . . . 20
6.3 Let's Build It . . . 20
6.3.1 Step 1: Add a timer . . . 20
6.3.2 Step 2: Make sure crashes are counted . . . 20
6.3.3 Step 3: Add a start screen . . . 21
6.3.4 Step 4: Add a result screen . . . 22

Quick Reference **23**

What's Next? **24**

Session 1: Where Is My Car?

Position and Direction

What We Already Know

You've done coordinates before in maths — like finding a square on a grid by going along and then up. "3 across, 2 up" — that kind of thing.

You also know about turns. A quarter turn is a right angle (90 degrees). A half turn is 180 degrees. A full turn is 360 degrees — you end up facing the same way you started.

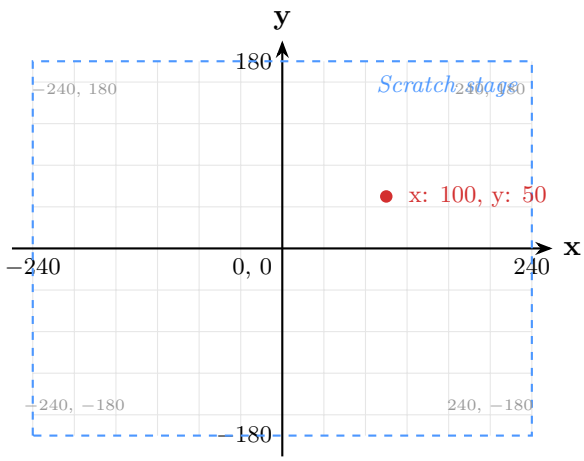
That's actually most of what you need to know to place a car on a screen and point it in a direction.

The New Idea: Position and Direction

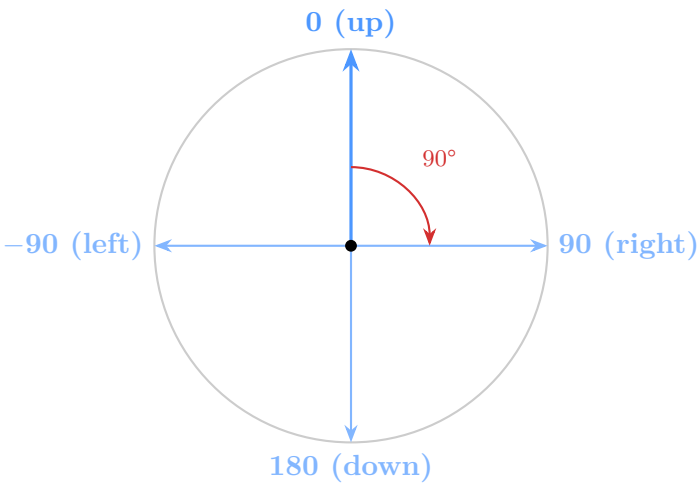
Everything in a game has two basic facts about it:

1. **Position** — *where* it is
2. **Direction** — *which way* it's facing

Position is just two numbers: how far across (left–right) and how far up–down. In Scratch, these are called **x** and **y**.



Direction is an angle — a number of degrees. In Scratch, direction works like a compass:



So if your car is at x: 100, y: 50 and its direction is 90, it's sitting in the right half of the screen, pointing to the right.

For grown-ups

Scratch's direction system is like a compass — 0 is north (up). This matches what children learn about compass directions in Year 3 geography. The x/y system maps directly to the coordinate grids they've used in maths, though Scratch extends into negative numbers. If your child hasn't met negatives yet, just say "minus means the other way" — left instead of right, or down instead of up.

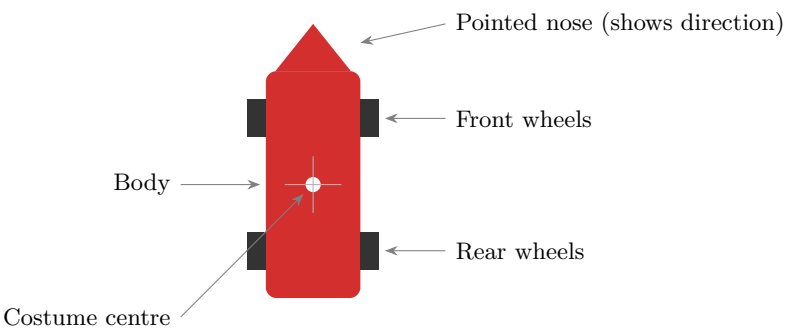
Let's Build It

Step 1: Create a new project

Open Scratch and click "Create". You'll see the cat sprite on the stage. We don't need the cat — right-click on it and choose "delete".

Step 2: Draw your car

Click the paintbrush icon ("Paint") to create a new sprite. We're going to draw a top-down racing car — a long, thin shape like an old-fashioned cigar-shaped race car.



1. Select the **oval tool** and draw a long, narrow oval in red (about 3 times taller than wide)
2. Add a small **triangle** at the top — the pointed nose that shows which way the car faces
3. Add four small **rectangles** for wheels (two at the back, two at the front)
4. Make sure the car points **up** (Scratch's default direction is 0 = up)

For grown-ups

The car's "costume centre" (the crosshair in the costume editor) should be in the middle of the car body. This is the point it rotates around. If it's off-centre, the car will wobble when it turns. Drag the costume to adjust this.

Step 3: Add movement code

Click on your car sprite, then go to the Code tab. Build this script:

Scratch code

```
when green flag clicked
go to x: (0) y: (0)
point in direction (0)
forever
  if <key [up arrow] pressed?> then
    move (3) steps
  end
  if <key [down arrow] pressed?> then
    move (-3) steps
  end
  if <key [left arrow] pressed?> then
    turn left (3) degrees
  end
  if <key [right arrow] pressed?> then
    turn right (3) degrees
  end
end
```

Where to find these blocks:

- `when green flag clicked` — **Events** (yellow)
- `go to x: y:` and `point in direction` — **Motion** (blue)
- `forever`, `if then` — **Control** (orange)
- `key pressed?` — **Sensing** (light blue)
- `move` and `turn` — **Motion** (blue)

Step 4: Try it!

Click the green flag. Use the arrow keys to drive around. **Up** moves forward, **Down** moves backward, **Left** and **Right** spin the car.

You've built the most basic driving game possible. The car knows *where* it is and *which way* it's facing, and you can change both.

Try it!

- Drive to each corner of the stage
- Try to drive in a straight line from left to right (hint: point the car to direction 90 first)
- Park on the exact centre (x: 0, y: 0) — check the x and y values below the stage

Want more?

Draw a road on the **backdrop**. Click the stage, then the "Backdrops" tab, and use the rectangle tool to draw grey roads on a green background. Now your car has somewhere to drive!

Session 2: Fast and Slow

Speed, acceleration, and braking

What We Already Know

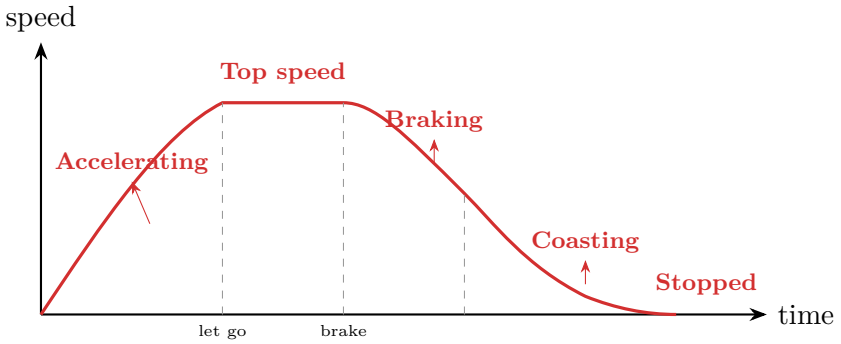
You know about measuring distances — centimetres, metres, kilometres. You know that some things are fast (a cheetah) and some things are slow (a snail). And you know from maths that bigger numbers mean more: 10 steps takes you further than 3 steps.

The New Idea: Speed

In Session 1, our car moved 3 steps every time. Real cars don't work like that. Real cars:

- **Speed up gradually** when you press the accelerator
- **Slow down gradually** when you let go
- **Brake** when you press the brake
- **Coast** to a stop — they don't stop instantly

Speed is just a number: "how far does the car move each moment?" If speed is 0, the car is still. If speed is 5, it zips along. If speed is −2, it goes backward.



Acceleration changes the speed. Press the accelerator, speed goes up. Brake, speed goes down.

Friction slows you down even when you're not braking. The road and air resistance drag on the car. In our game, we shrink the speed by 5% each moment, so the car naturally coasts to a stop.

For grown-ups

We're keeping this intuitive — speed as "steps per frame," acceleration as "speed change per frame." Friction is simplified to a multiplier (`speed` × `0.95` each frame), which mimics drag well for a game. The formal $v = d/t$ and the distinction between static and kinetic friction aren't needed here — the key insight is just "things slow down on their own."

Let's Build It

Step 1: Create a speed variable

Go to **Variables** (orange) and click "Make a Variable". Call it `speed`. Select "For this sprite only".

Step 2: Update the movement code

Replace your old script with this:

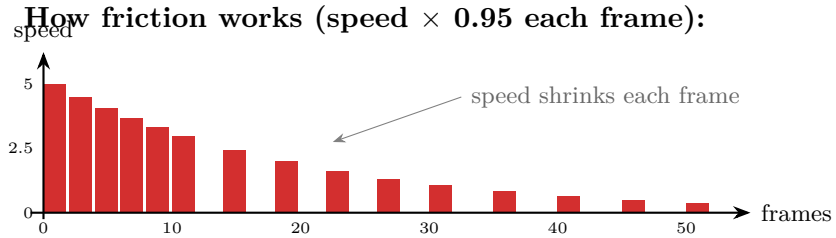
Scratch code

```
when green flag clicked
go to x: (0) y: (0)
point in direction (0)
set [speed] to (0)
forever
  if <key [up arrow] pressed?> then
    change [speed] by (0.3)
  end
  if <key [down arrow] pressed?> then
    change [speed] by (-0.5)
  end
  if <key [left arrow] pressed?> then
    turn left (3) degrees
  end
  if <key [right arrow] pressed?> then
    turn right (3) degrees
  end
  move (speed) steps
  set [speed] to ((speed) * (0.95))
end
```

What changed:

- `speed` starts at 0 (car begins stationary)
- Up arrow increases `speed` by 0.3 (gentle acceleration)
- Down arrow decreases `speed` by 0.5 (braking is stronger — like a real car)
- `move (speed) steps` — the car moves by the current speed
- `speed × 0.95` — **friction!** Speed shrinks 5% each frame, so the car coasts to a stop

How friction works (speed × 0.95 each frame):



Step 3: Add a speed limit

Cars can't go infinitely fast. Add this inside the `forever` loop, after the key checks but before `move`:

Scratch code

```
if <(speed) > (5)> then
  set [speed] to (5)
end
if <(speed) < (-3)> then
  set [speed] to (-3)
end
```

This caps forward speed at 5 and reverse at 3 (reversing is always slower).

For grown-ups

The × `0.95` is the key physics trick. Speed decays exponentially — fast speeds drop quickly, slow speeds linger. If your child asks "why 0.95?", try changing it: 0.8 = lots of friction (like sand), 0.99 = almost none (like ice). Great way to build intuition.

Try it!

- Accelerate to top speed, then let go. How far does the car coast?
- Try to stop on an exact spot — it's harder now!
- Hold down arrow while moving forward — feel how braking works
- What happens if you hold down arrow when stopped? (It reverses!)

Want more?

Make the `speed` variable visible on stage (tick its checkbox). Instant speedometer! For a fancier one, create a bar sprite whose size follows the speed value.

Session 3: Turning the Wheel

Steering and turning radius

What We Already Know

You know about turns — quarter turns (right angles), half turns, full turns. You know that turning the steering wheel makes the front wheels point left or right, and that's what makes a car turn.

Here's something you might have noticed: if a car is standing still and you turn the steering wheel, the car doesn't go anywhere. The wheels just point sideways. The car only *actually turns* when it's moving.

The New Idea: Steering and Turning Radius

How fast you're going changes how sharply you can turn.

Think about riding a bicycle:

- Going **slowly**: you can turn really tightly, almost spinning on the spot
- Going **fast**: you have to make a big, wide turn
- **Standing still**: you can turn the handlebars but the bike doesn't change direction

Slow = tight turn **Fast = wide turn**

This is called the **turning radius**. Slow = tight turns. Fast = wide turns.

This is exactly why parking needs slow speed. You need tight turns to fit into a space, and you can only make tight turns when you're going slowly.

For grown-ups

The real physics involves the relationship between velocity, steering angle, and the resulting circular arc ($R = L/\tan(\theta)$ where L is wheelbase length). We simplify to "turning amount scales with speed," which captures the essential insight. The key idea — that turning effectiveness depends on forward motion — is genuinely important and often surprises children and adults alike.

Let's Build It

Step 1: Create a steering variable

Make a new variable called `steering` (for this sprite only). It represents the steering wheel position: negative = left, positive = right, zero = straight.

Step 2: Update the steering code

Replace the left/right arrow handling and the turn line:

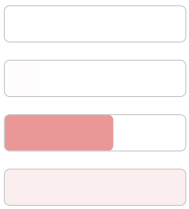
Scratch code

```
set [steering] to (0)
  ...inside forever loop...
  if <key [left arrow] pressed?> then
    set [steering] to (-5)
  else
    if <key [right arrow] pressed?> then
      set [steering] to (5)
    else
      set [steering] to (0)
    end
  end
  turn right ((steering) * ((speed) / (3))) degrees
```

The key line: `turn right ((steering) × ((speed) / (3))) degrees`

This means: turn by the steering amount, multiplied by how fast you're going.

What happens at different speeds (steering = 5):



Stopped: $5 \times 0 =$ **no turning**

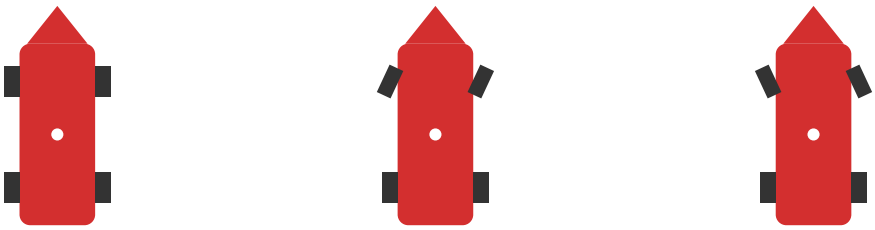
Creeping: $5 \times 0.33 =$ **tiny turn**

Medium: $5 \times 1 =$ **normal turn**

Top speed: $5 \times 1.67 =$ **bigger turn per step**

Step 3: Add a wheel indicator

Create three costumes for your car:



straight **left** **right**

Add this inside the `forever` loop:

Scratch code

```
if <(steering) < (0)> then
  switch costume to [left]
else
  if <(steering) > (0)> then
    switch costume to [right]
  else
    switch costume to [straight]
  end
```

```
end
```

Now you can see the wheels turn when you steer!

Try it!

- Drive in a circle at slow speed. Now try at top speed — what happens?
- Try to drive in a perfect square: forward, right angle turn, forward... (slow down for corners!)
- Coast forward and steer — the turning gets tighter as you slow down!

Want more?

Try changing the steering value from 5 to 8 or 3. Racing drivers call this "steering sensitivity" — find what feels best for your game.

Session 4: Walls and Bumps

Collision detection

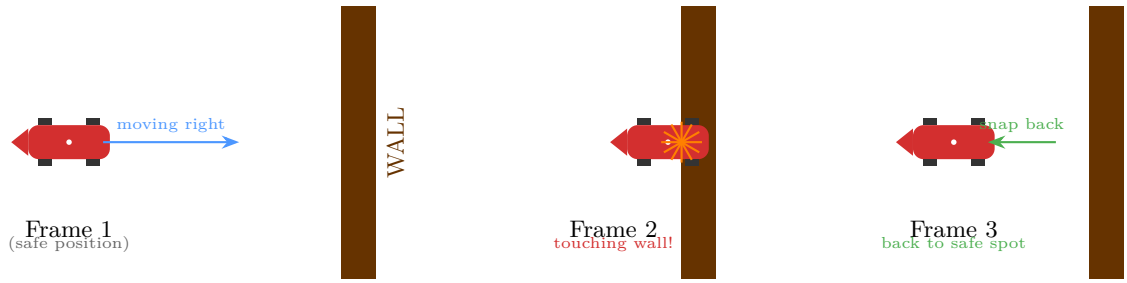
What We Already Know

In science, you've learned about **forces** — pushes and pulls. When you push a toy car into a wall, the wall pushes back. The car stops. Two solid things can't be in the same place at the same time.

The New Idea: Collision

A **collision** is when two objects touch or overlap. In our game, we check every single frame: "is the car touching a wall?" If so:

1. **Push back** to the last safe position
2. **Stop** (set speed to 0)
3. **Bonk!** (play a bump sound)



For grown-ups

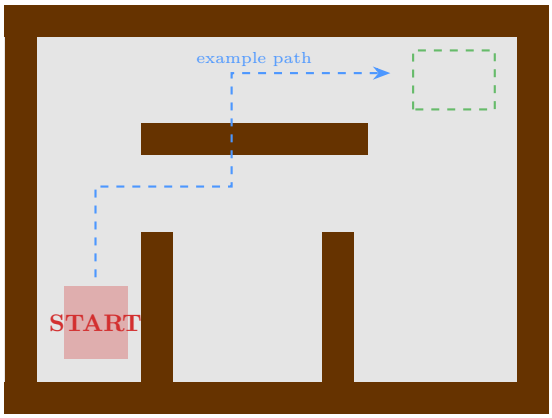
Real game physics engines use sophisticated collision detection (separating axis theorem, impulse resolution). In Scratch, we use colour-based collision, which is simpler but works well. The "push back" is important — without it, the car gets stuck inside the wall. We remember the last safe position and snap back to it each time a collision is detected.

Let's Build It

Step 1: Design the course

Click on the **Stage**, then the **Backdrops** tab. Draw a course:

1. Fill the backdrop with **light grey** (road)
2. Use **dark brown** (`#663300`) to draw walls: border walls and some inner walls to create corridors
3. Leave a clear starting area and a path to where the parking spot will go



Important: Use *one specific colour* for all walls. We detect this colour to know when the car has hit a wall.

Step 2: Add position memory

Create two new variables (for this sprite only):

- `last x`
- `last y`

Step 3: Update the code

Add these lines to your car's `forever` loop. **Before** any movement:

Scratch code

```
set [last x] to (x position)
set [last y] to (y position)
```

After the `move` block, add collision detection:

Scratch code

```
if <touching color (#663300)?> then
  go to x: (last x) y: (last y)
  set [speed] to (0)
  start sound [Boing]
end
```

This checks: "Am I touching the wall colour?" If yes: snap back, stop, bonk.

For grown-ups

To pick the right colour in the `touching color` block, click the colour swatch, then click the eyedropper tool, then click on the wall colour on your backdrop. The "Boing" sound is in Scratch's sound library — go to the Sounds tab and click the speaker icon to browse and add it.

Also change the starting position to somewhere sensible on your course:

Scratch code

```
go to x: (-180) y: (-120)
```

Try it!

- Drive the full course without hitting a single wall
- Try hitting a wall at top speed vs. low speed
- Find the tightest corner. Can you get around it? (Remember: slow = tight turns!)

Want more?

Add a crash counter! Create a `crashes` variable (for all sprites). Set it to 0 on green flag. Inside the collision `if` block, add `change [crashes] by (1)`. Can you finish the course with zero crashes?

Session 5: Park It!

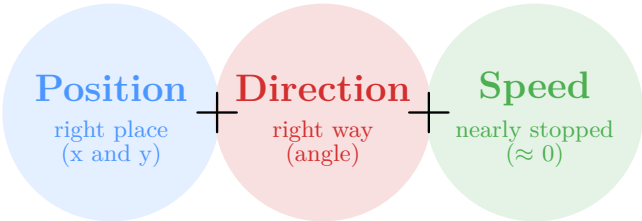
The parking challenge

What We Already Know

You've fitted shapes into spaces before — jigsaw puzzles, shape sorters, tidying a shelf. You know that to fit something into a space, it needs to be the right size, in the right place, and pointing the right way.

The New Idea: Parking as a Physics Puzzle

Parking is hard because you need **three things right at the same time:**



All three at once = PARKED!

Each one on its own is easy. But doing all three *together* — that's the challenge. And because you can only turn well at low speed (Session 3), you have to do the hard part at exactly the speed where your controls are most sensitive.

That's why even grown-ups find parking tricky!

For grown-ups

This connects to the mathematical idea of *degrees of freedom*. The car has three variables to satisfy simultaneously (x, y, angle), which is what makes parking a challenging control problem. You might mention that real self-driving cars find parking one of the hardest tasks too!

Let's Build It

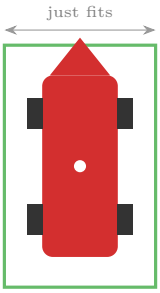
Step 1: Create the parking spot

Create a new sprite (paintbrush icon). Draw a rectangle **outline only** — no fill. Use bright **green** or **yellow**. Size it so the car just barely fits inside with a little room to spare.

Name this sprite "ParkingSpot" and position it:

Scratch code

```
when green flag clicked
go to x: (150) y: (100)
point in direction (0)
```



Step 2: Add parking detection

Add this script to the **ParkingSpot** sprite:

Scratch code

```
when green flag clicked
go to x: (150) y: (100)
point in direction (0)
set [parked] to (0)
forever
  if <<touching [Car]?> and
      <([speed] of [Car]) < (0.5)>> then
    if <<([speed] of [Car]) > (-0.5)> and
        <((abs of (([direction] of [Car]) - (0))) < (20))>> then
      set [parked] to (1)
      start sound [Collect]
      say [Parked!] for (2) seconds
      stop [all]
    end
  end
end
```

That's a lot of conditions! Here's what each one checks:

1. `touching [Car]?` — car overlaps the parking spot

2. `speed < 0.5` and `speed > -0.5` — car is nearly stopped

3. `abs(direction - 0) < 20` — car faces roughly the right way

↓

All true at once = PARKED!

For grown-ups

The `abs` block is in the **Operators** category (green), under the dropdown that starts with "sqrt". If your child asks what "abs" means, say: "it throws away the minus sign — it tells us how far from zero, whether plus or minus." The tolerance of 20 degrees is adjustable — start generous, tighten later for more challenge.

Step 3: Add a happy sound

On the ParkingSpot sprite, go to the **Sounds** tab, click the speaker icon, and add a celebratory sound — "Collect", "Win", or "Tada".

Step 4: Try it!

Drive to the spot. Line up. Slow down. Match the angle. When all three conditions are met — **PARKED!** Happy sound, message, game stops.

This is the moment. This is what we've been building towards.

Try it!

- Park the car. How many attempts does it take?
- Try approaching from different angles — which approach is easiest?
- Can you park with zero crashes?
- Try driving in fast — the speed check stops you from "cheating"!

Want more?

Add a second parking spot in a trickier location — maybe between two walls. Duplicate the ParkingSpot sprite and reposition it.

Session 6: Make It a Real Game

Polish, scoring, and the full experience

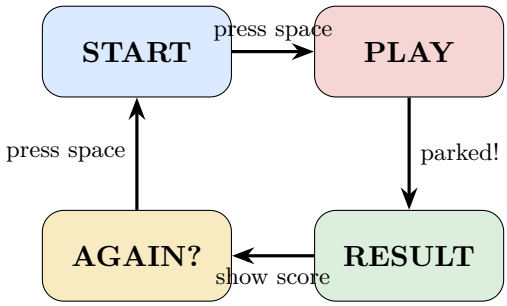
What We Already Know

You know about counting and timing. You know what a "personal best" is. You know that the best games have a beginning, a middle, and an end.

Right now our game just... starts. No "ready, set, go." No score. No replay. Let's fix that.

The New Idea: The Game Loop

Every game follows a loop:



This isn't physics — it's *game design*. But it's what turns a physics simulation into a game.

For grown-ups

The game loop is a fundamental concept in computer science. Every game your child has played uses this pattern. The timer and crash counter together create a scoring tension — speed vs. care — which is what makes the game interesting and replayable.

Let's Build It

Step 1: Add a timer

Scratch has a built-in timer. Add to the car's setup (right after starting):

Scratch code

```
reset timer
```

Update the ParkingSpot's parking message to show the time:

Scratch code

```
say (join [Parked in ] (join (round (timer)) [ seconds!]))
  for (3) seconds
```

Step 2: Make sure crashes are counted

If you haven't already, create a `crashes` variable (for all sprites). In the car's setup:

Scratch code

```
set [crashes] to (0)
```

In the collision `if` block:

Scratch code

```
change [crashes] by (1)
```

Step 3: Add a start screen

Create a new sprite called "StartScreen". In the costume editor, write "PARK LIKE A PRO" in large text, with "Press SPACE to start" underneath.

Scratch code

```
when green flag clicked
show
go to x: (0) y: (0)
go to [front] layer
wait until <key [space] pressed?>
hide
broadcast [start game]
```

Now update the **car** and **ParkingSpot** sprites. Each gets *two* scripts:

Car sprite:

Scratch code

```
when green flag clicked
hide
go to x: (-180) y: (-120)
point in direction (0)

when I receive [start game]
show
set [speed] to (0)
set [steering] to (0)
set [crashes] to (0)
reset timer
forever
  ...(all your driving code)...
end
```

ParkingSpot sprite:

Scratch code

```
when green flag clicked
show
go to x: (150) y: (100)

when I receive [start game]
set [parked] to (0)
forever
  ...(parking detection code)...
```

```
  ...(replace "stop [all]" with:)
  broadcast [game over]
  stop [other scripts in sprite]
end
```

Step 4: Add a result screen

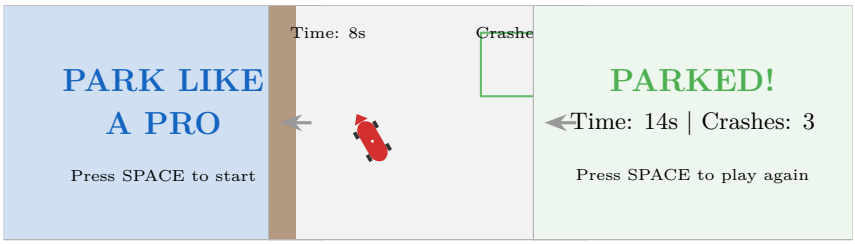
Create a "ResultScreen" sprite:

Scratch code

```
when green flag clicked
hide

when I receive [game over]
show
go to [front] layer
say (join (join [Time: ] (join (round (timer)) [s]))
         (join [ | Crashes: ] (crashes)))
wait (3) seconds
say [Press SPACE to play again!]
wait until <key [space] pressed?>
hide
broadcast [start game]
```

Now the game loops: start → play → result → play again!



Start screen **Playing** **Result screen**

Try it!

- Play 3 rounds. What's your best time?
- Try for 0 crashes AND a fast time — speed vs. care!
- Show someone else. Watch them play. Is the game fun for them?
- What would you change to make it harder? Easier?

Want more?

- Add more parking spots — park in all of them to win
- Add a "best time" variable that remembers your record across rounds
- Create different course layouts for different levels

Quick Reference: Scratch Blocks

Block	Category	What it does
`when green flag clicked`	Events	Starts the script
`when I receive`	Events	Starts on broadcast
`broadcast`	Events	Sends message to all sprites
`go to x: y:`	Motion	Sets position
`point in direction`	Motion	Sets facing angle
`move (n) steps`	Motion	Moves forward/backward
`turn (n) degrees`	Motion	Rotates the sprite
`x position / y position`	Motion	Current position
`direction`	Motion	Current angle
`forever`	Control	Loops forever
`if <> then / else`	Control	Checks a condition
`wait until <>`	Control	Pauses until true
`stop [all]`	Control	Stops everything
`key pressed?`	Sensing	Is a key held down?
`touching [sprite]?`	Sensing	Sprite overlap check
`touching color?`	Sensing	Touching a colour?
`timer / reset timer`	Sensing	Built-in stopwatch
`set / change variable`	Variables	Update stored numbers
`join`	Operators	Combine text
`round`	Operators	Round a number
`abs of`	Operators	Distance from zero
`show / hide`	Looks	Visible/invisible
`switch costume`	Looks	Change appearance
`say for secs`	Looks	Speech bubble
`go to front layer`	Looks	Put on top
`start sound`	Sound	Play a sound effect

What's Next?

You've built a real game! Here are ideas if you want to keep going:

Session 7: Reverse! Put the car in reverse gear. When speed is negative, steering works *backwards* — turning left makes the back go right. This is why reversing into a parking space is so tricky. Try adding parallel parking!

Session 8: Traffic! Add other cars. Start with parked cars as obstacles. Then try making one drive back and forth. Now you have to time your moves!

Session 9: Different Cars. What if the car was longer? Shorter? Longer cars are harder to park (bigger turning radius). Short cars are easier. Try making a stretched "limo" version and see the difference. That's physics in action.

Made with curiosity and Scratch. Happy parking!